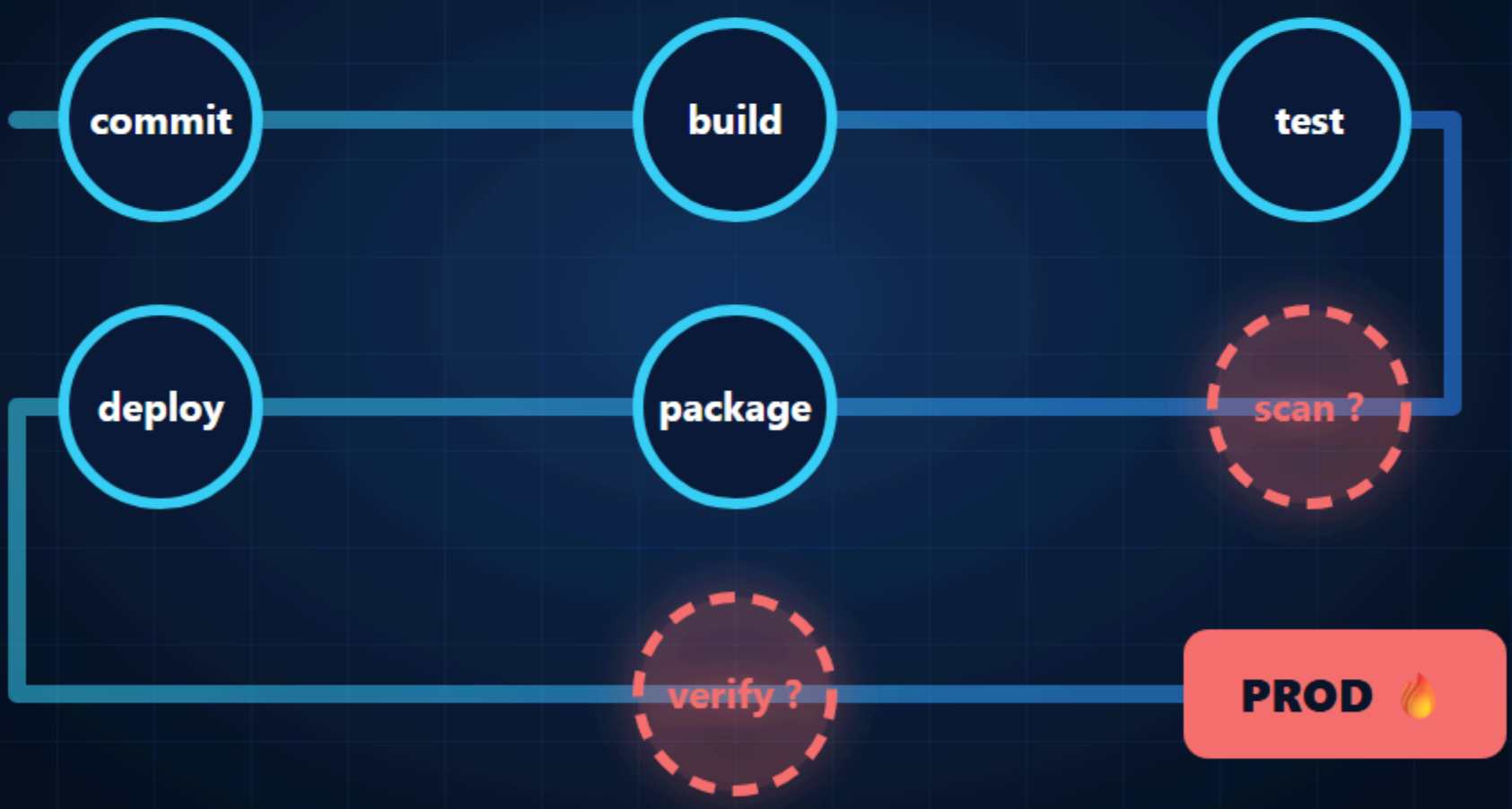
DAY 6 · CI/CD FUNDAMENTALS

Most pipeline failures aren't build failures. They're missing stages.



7 stages · 2 skipped · 1 incident

INFOGRAPHIC 1/3 CI/CD FUNDAMENTALS

From commit to production: the 7 stages and what each must catch

Continuous integration proves every change builds and passes tests. Continuous delivery keeps every passing build releasable. Each stage is a gate that stops a specific kind of mistake.

01

Commit: small changes to a protected trunk

Short-lived branches, a pull request, required reviews and required status checks on main.

Why: small changes are easy to review, test and roll back; big merges hide bugs.

```
Branch protection on main: require PR review + required checks "build", "test", "scan"
```

02

Build once, from a clean, pinned environment

One build per commit, with locked dependency versions, tagged with the commit SHA.

Why: rebuilding per environment means prod runs something that was never tested.

```
docker build -t "$REGISTRY/app:$GIT_SHA" .
```

03

Test: fastest feedback first

Unit tests, then integration, then a few end-to-end tests. Fail fast and stop the line.

Why: developers fix what they hear about in minutes; they ignore what arrives hours later.

```
make test-unit && make test-integration   # quarantine flaky tests, never ignore them
```

04

Scan: code, dependencies, secrets, IaC, image

Static analysis, dependency (SCA), secret, infrastructure-as-code and container image scans.

Why: the most expensive mistakes (a leaked key, a known-exploited library) pass every test.

```
trivy fs --scanners vuln,secret,misconfig --severity HIGH,CRITICAL --exit-code 1 .
```

	Scan	Catches	Runs on
SAST	Static analysis of your code	Injection, unsafe APIs, weak crypto use	Every PR
SCA	Dependency analysis	Libraries with known vulnerabilities	Every PR + daily
Secrets	Pattern + entropy matching	Keys and tokens in code and history	Pre-commit + PR
IaC	Terraform, Kubernetes, Helm	Public buckets, open ports, no encryption	Every PR
Image	Built container image	Vulnerable OS packages, root user	Build + registry

Gate rule: block on a secret or a HIGH/CRITICAL finding; allow exceptions only with an owner, a reason and an expiry date.

INFOGRAPHIC 2/3 PACKAGE, DEPLOY, VERIFY, MEASURE

Ship the same artifact everywhere, and prove it works

05

Package: immutable, signed, with an SBOM

Push once to a registry, refer to it by digest, attach an SBOM and a signature.

Why: you can prove what is running, where it came from, and what is inside it.

```
syft "$IMAGE" -o spdx-json > sbom.spdx.json && cosign sign --yes "$REGISTRY/app@$DIGEST"
```

06

Deploy: promote the digest, progressively

Staging, then production behind an approval; canary or blue/green; keyless OIDC credentials.

Why: a bad release reaches a slice of users, not all of them; no cloud keys to leak.

```
kubectl set image deployment/app app="$REGISTRY/app@$DIGEST"
```

07

Verify, and roll back automatically

Wait for the rollout, run smoke tests, watch error rate and latency against your SLO.

Why: "deployed" is not "working". Verification turns an outage into a rollback.

```
kubectl rollout status deployment/app --timeout=5m || kubectl rollout undo
deployment/app
```

08

Measure the pipeline itself

Track DORA's delivery metrics: deployment frequency, lead time for changes, change fail rate, recovery time.

Why: they show whether a pipeline change made delivery faster and safer, or just different.

```
Per deploy, record: commit time, deploy time, success/failure, time to restore
```

	AWS	Azure	Google Cloud
Native CI/CD	CodeBuild, CodePipeline, CodeDeploy	Azure Pipelines	Cloud Build, Cloud Deploy
Registry	Amazon ECR	Azure Container Registry	Artifact Registry
Keyless CI auth	IAM OIDC identity provider + role	Entra workload identity federation	Workload Identity Federation

INFOGRAPHIC 3/3 QUALITY GATES

Every stage needs a gate that can fail the build

	The gate	Stop the pipeline when
Commit	Review + required checks	A required check fails or review is missing
Build	Reproducible, pinned build	Compile errors, lockfile drift, unpinned base image
Test	Unit → integration → e2e	Any failure; coverage of changed code drops
Scan	SAST, SCA, secrets, IaC, image	A secret is found; HIGH/CRITICAL finding without an approved exception
Package	Digest, SBOM, signature	Signing fails; the image isn't referenced by digest
Deploy	Environment approval, policy	Unsigned image; missing approval for production
Verify	Rollout status, smoke, SLO	Rollout times out; error rate or latency breaches the SLO

✓ Must know

CI ≠ CD: integration is about merging safely; delivery is about always being releasable

Build once, promote many: the digest tested in staging is the one in prod

Pipeline as code, reviewed like application code

Pin third-party actions/plugins to a commit SHA, not a moving tag

Least privilege: deploy jobs get short-lived, scoped credentials

✗ Pitfalls

Rebuilding the image for each environment

Long-lived cloud keys stored as CI secrets

Scans that only warn, so everyone ignores them

"latest" tags in production manifests

No verify stage: success means "the command exited 0"

Gate order	**Cheapest first:** lint and format → unit tests → secret and dependency scans → build image → integration tests → image scan → e2e. Fail at the first gate that can find the problem.
Who decides prod	**Continuous delivery:** a person approves the production deploy. **Continuous deployment:** the verify stage decides, so it must be automated and trusted.
Pipeline health	Track how long the pipeline takes and how often it fails for reasons unrelated to the change; both erode trust faster than any outage.

WHAT & WHY

CI, continuous delivery and continuous deployment are not the same

Continuous integration (CI)

Every change is merged to a shared trunk often, and every merge is built and tested automatically. The goal: the trunk always works.

Continuous delivery (CD)

Every change that passes the pipeline is releasable. Production deploys are a business decision, often one approval away.

Continuous deployment

Every change that passes the pipeline goes to production automatically, with no human gate. It needs strong tests and automated verification.

Why it matters: most pipeline incidents don't come from a broken build. They come from a stage that was never there: no scan, no verify, no rollback.

THE MAP

The 7 stages of a reliable pipeline

01	**Commit:** small change, pull request, review and required checks on a protected trunk
02	**Build:** once per commit, in a clean environment, with pinned dependencies
03	**Test:** unit, integration, then a few end-to-end tests, fastest first
04	**Scan:** code, dependencies, secrets, infrastructure-as-code and the image
05	**Package:** an immutable artifact, referenced by digest, with an SBOM and a signature
06	**Deploy:** promote that same artifact through environments, progressively
07	**Verify:** rollout status, smoke tests and SLO checks, with automatic rollback

Exercise: write your pipeline's jobs next to these seven. The empty row is your next improvement.

STAGE 1 · COMMIT

Protect the trunk, keep changes small

- → **Short-lived branches** (hours to a couple of days) merged into main, or trunk-based development with feature flags
- → **Branch protection:** pull request required, at least one review, required status checks, no force-push to main
- → **CODEOWNERS** for pipeline files, infrastructure code and security-sensitive paths
- → **Pre-commit hooks** for formatting, linting and secret detection, so trivial failures never reach CI
- → **Conventional commit messages** or linked tickets, so a release can be traced back to its changes

> **Rule of thumb:** if a pull request is too big to review properly, it's too big to roll back safely.

STAGE 2 · BUILD

Build once, reproducibly

- → **One build per commit;** every later stage reuses its output
- → **Lock dependencies** (package-lock.json, poetry.lock, go.sum) and pin base images by digest
- → **Ephemeral, clean runners:** nothing left over from the previous job
- → **Cache dependencies,** never build outputs that skip a real build
- → **Tag with the commit SHA,** never only "latest"

```
# Pin the base image by digest, use a multi-stage build
FROM golang:1.23@sha256:<digest> AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -o /app ./cmd/app

FROM gcr.io/distroless/static@sha256:<digest>
COPY --from=build /app /app
ENTRYPOINT ["/app"]
```

STAGE 3 · TEST

Fast feedback first, confidence last

Unit
Many, fast, no network. Run on every push.

Integration
Real dependencies in containers: database, queue, APIs.

End-to-end
Few, critical user journeys only. Slow and brittle when overused.

→ **Order by speed:** fail on the cheapest test that can find the problem

→ **Parallelize** test suites and split by timing data

→ **Flaky tests:** quarantine with an owner and a deadline; never "just re-run"

→ **Test the migration** path for databases, not just the new schema

→ **Contract tests** between services catch breaking API changes before deploy

> **Signal:** if engineers routinely re-run a failed pipeline without reading it, your tests have lost trust.

STAGE 4 · SCAN

Catch what tests can't

	Finds	Example tools
SAST	Injection, unsafe APIs in your own code	CodeQL, Semgrep
SCA	Known-vulnerable dependencies	Dependabot, Trivy, OSV-Scanner
Secrets	Keys and tokens in code or history	Gitleaks, Trivy, push protection
IaC	Public buckets, open security groups	Checkov, Trivy (misconfig)
Image	Vulnerable OS packages in the image	Trivy, Grype, registry scanning

```
trivy fs --scanners vuln,secret,misconfig \
  --severity HIGH,CRITICAL --exit-code 1 .
trivy image --severity HIGH,CRITICAL --exit-code 1 "$IMAGE"
```

Make it a gate: fail on a defined severity, with a documented, time-limited exception process. Warnings nobody acts on are noise.

STAGE 5 · PACKAGE

Immutable, signed, traceable artifacts

→ **Push once** to your registry and refer to the image by **digest** (sha256), which can't be changed

→ **SBOM:** a machine-readable list of what's inside (SPDX or CycloneDX)

→ **Signature:** proves the image came from your pipeline and wasn't swapped

→ **Provenance:** records how and where it was built; SLSA defines levels for this

```
# SBOM, keyless signature (Sigstore), then verify
syft "$IMAGE" -o spdx-json > sbom.spdx.json
cosign sign --yes "$REGISTRY/app@$DIGEST"
cosign verify "$REGISTRY/app@$DIGEST" \
  --certificate-identity-regexp 'https://github.com/acme/app/.*' \
  --certificate-oidc-issuer https://token.actions.githubusercontent.com
```

STAGE 6 · DEPLOY

Promote the digest, not a rebuild

- → **Same digest** moves dev → staging → production; only configuration changes
- → **Environments with protection:** required reviewers and branch rules for production
- → **Keyless auth:** CI exchanges its OIDC token for short-lived cloud credentials (AWS IAM OIDC provider, Entra workload identity federation, Google Workload Identity Federation)
- → **Progressive delivery:** canary (a small share of traffic first) or blue/green (switch, keep the old one warm)
- → **Concurrency control:** one production deploy at a time
- → **Database changes** backward compatible, so the previous version can still run

> **Admission policy:** let the cluster refuse unsigned images or tags instead of digests, so a manual kubectl apply can't bypass the pipeline.

STAGE 7 · VERIFY

"Deployed" is not "working"

01 **Rollout status:** wait until new pods are ready, with a timeout

02 **Smoke tests:** hit health and one or two critical endpoints

03 **SLO check:** compare error rate and latency with the previous version for a fixed window

04 **Decide automatically:** promote, or roll back without waiting for a human

```
kubectl rollout status deployment/app --timeout=5m \
  || kubectl rollout undo deployment/app
curl -fsS https://app.example.com/healthz
```

Practice the rollback on a normal day. The first time you run it shouldn't be during an incident.

PUTTING IT TOGETHER

A pipeline skeleton (GitHub Actions)

```yaml
name: delivery
on:
  push:
    branches: [main]
permissions:
  contents: read
  id-token: write              # OIDC to the cloud, no stored keys
jobs:
  ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@<commit-sha>   # pin to a SHA
      - run: make build test
      - run: trivy fs --scanners vuln,secret,misconfig --exit-code 1 .
      - run: make image push sbom sign   # outputs the digest
  deploy-staging:
    needs: ci
    environment: staging
    runs-on: ubuntu-latest
    steps:
      - run: make deploy verify ENV=staging
deploy-prod:
  needs: deploy-staging
  environment: production    # required reviewers
  concurrency: production
  runs-on: ubuntu-latest
  steps:
    - run: make deploy verify ENV=prod
```

MEASURE

Is the pipeline making delivery better?

Deployment frequency
How often you successfully release to production

Lead time for changes
Time from commit to running in production

Change fail rate
Share of deployments that cause a failure needing remediation

Failed deployment recovery time
How long it takes to recover from a failed deployment

> **DORA's four keys** balance speed (the first two) against stability (the last two). Improve one without hurting the other.

→ Record commit time, deploy time and outcome for every production deploy

→ Review the trend monthly, not per team leaderboard

PITFALLS

8 pipeline mistakes that cause incidents

→ **Rebuilding per environment:** prod runs an untested artifact

→ **Long-lived cloud keys** stored as CI secrets

→ **Unpinned third-party actions or plugins** that can change under you

→ **Scans that only warn** and never block

→ **"latest" tags** in production manifests

→ **No verify stage:** success means the deploy command exited 0

→ **Flaky tests re-run until green**

→ **Manual hotfixes** that bypass the pipeline and never get back into it

CHECKLIST

CI/CD pipeline checklist

- ☐ Main is protected: PR, review, required checks
- ☐ One reproducible build per commit, tagged with the SHA
- ☐ Tests ordered fastest first; flaky tests have owners
- ☐ SAST, SCA, secret, IaC and image scans gate the build
- ☐ Images referenced by digest, with an SBOM and a signature
- ☐ Same digest promoted through every environment
- ☐ OIDC short-lived credentials; no stored cloud keys
- ☐ Production approval and one deploy at a time
- ☐ Automated verification and rollback
- ☐ DORA metrics recorded for every production deploy

HANDS-ON LAB

Try it: map your pipeline to the 7 stages

01 Write each job of your current pipeline next to the stage it covers: commit, build, test, scan, package, deploy, verify

02 Pick the missing or weakest stage

03 Add it in a sandbox repository first, as a gate that can fail the build

04 Break it on purpose (commit a fake key, deploy a bad image) and confirm the pipeline stops

```
# In a throwaway repo: prove the scan gate works
echo "AWS_SECRET_ACCESS_KEY=$(openssl rand -base64 30)" > leak.env  # random, fake
trivy fs --scanners secret --exit-code 1 .   # must fail
# Prove the rollback works
kubectl set image deployment/app app=nginx:does-not-exist
kubectl rollout status deployment/app --timeout=60s \
  || kubectl rollout undo deployment/app
```

Teardown: delete leak.env, the throwaway repository and the test deployment.

FURTHER READING

Go deeper: official references

DORA: Continuous delivery
dora.dev/capabilities/continuous-delivery/

DORA: Software delivery performance metrics
dora.dev/guides/dora-metrics-four-keys/

AWS Well-Architected: DevOps Guidance
docs.aws.amazon.com/wellarchitected/latest/devops-guidance/devops-guidance.html

SLSA: Supply-chain Levels for Software Artifacts
slsa.dev/

Sigstore: Signing containers with Cosign
docs.sigstore.dev/cosign/signing/signing_with_containers/

Docs change. Always check the current version before you design.

YOUR TURN

Which stage is missing from your pipeline today: scan, package or verify?

Tell me in the comments. I read and reply to every one.

Follow Deji Ajishe

Day 6 of my DevOps series

♻️ Repost to help an engineer on your team

linkedin.com/in/deji-ajishe-64a35722a